EJERCICIOS PRÁCTICOS - INTRODUCCIÓN A JAVASCRIPT

INFORMACIÓN DE CONTACTO DEL INSTRUCTOR

Ítalo Ravina
Discord: italoravina_87123
LinkedIn: Ítalo Ravina Clemente | LinkedIn

INSTRUCCIONES GENERALES

- **No se permite el uso de librerías externas**. Todo el código debe ser creado por el alumno.
- Cada ejercicio debe estar en un **archivo JavaScript separado** con el nombre indicado.
- Usa **nombres de variables y funciones descriptivos** (preferiblemente en inglés).
- **Comenta tu código** cuando sea necesario para explicar la lógica.
- Prueba tu código con diferentes casos para asegurar que funciona correctamente.

Los ejercicios 1 a 4 son OBLIGATORIOS. Puedes elegir hacer el ejercicio 5 o 6. El ejercicio Mini-Hotel es opcional.

EJERCICIO 1: Presentación

Crear un objeto que represente tu perfil como desarrollador. Debe tener los siguientes campos: nombre, apellidos, un array con los temas del bootcamp que ya conoces, si tienes experiencia previa programando (boolean), un array de objetos con información de alguna red social que tengas (Linkedin, Github…), ese objeto debe tener el nombre y el link.

EJERCICIO 2: Cazador de bugs

2.1 Un cliente necesita una función que cuente cuántas vocales hay en un texto, pero nos dice que **siempre cuenta de menos**. No nos da más detalles, solo el código.

1. **Identifica el bug** - ¿Qué está mal?
2. **Explica el problema** - ¿Por qué no cuenta todas las vocales?
3. **Arregla el código** - Corrígelo para que funcione correctamente
4. **Prueba con varios casos** - Verifica con diferentes textos

```
function countVowels(text) {
  const vowels = ["a", "e", "i", "o", "u"];
  let counter = 0;

  for (let i = 0; i < text.length; i++) {
    if (vowels.includes(text[i])) {
      counter++;
    }
  }

  return counter;
}

const phrase = "Antes no programaba. Ahora si!";
const result = countVowels(phrase);
console.log(`La frase tiene ${result} vocales`);
```

2.2 Un cliente necesita una función que duplique cada número en un array (multiplicar por 2), pero nos dice que **el array original también se está modificando** y no quiere eso.

1. **Identifica el bug** - ¿Por qué se modifica el array original?
2. **Explica el concepto** - ¿Qué significa "pasar por referencia"?
3. **Arregla el código** - Haz que NO modifique el original
4. **Bonus**: Reescríbelo usando `.map()`

```
function duplicateNumbers(numbers) {
  for (let i = 0; i < numbers.length; i++) {
    numbers[i] = numbers[i] * 2;
  }
  return numbers;
}

const original = [1, 2, 3, 4, 5];
const duplicated = duplicateNumbers(original);

console.log("Original:", original);
console.log("Duplicated:", duplicated);
```

EJERCICIO 3: Transformaciones

Un cliente tiene un array de desarrolladores y nos pide extraer información específica usando métodos de arrays. Quiere obtener los programadores que tengan como habilidad JavaScript y un listado de los proyectos en los que están trabajando

Datos proporcionados:

```
const data = [
  {
    id: 1,
    nombre: 'Juan',
    habilidades: ['JavaScript', 'HTML', 'CSS'],
    proyectos: [
      { id: 1, nombre: 'Proyecto 1' },
      { id: 2, nombre: 'Proyecto 2' }
    ]
  },
  {
    id: 2,
    nombre: 'María',
    habilidades: ['Python', 'SQL', 'Django'],
    proyectos: [
      { id: 3, nombre: 'Proyecto 3' },
      { id: 4, nombre: 'Proyecto 4' }
    ]
  },
  {
    id: 3,
    nombre: 'Miriam',
    habilidades: ['UX', 'Figma', 'HTML', 'JavaScript'],
    proyectos: [
      { id: 5, nombre: 'Proyecto 1' },
      { id: 6, nombre: 'Proyecto 4' }
    ]
  }
];
```

TAREAS:

1. Filtrar desarrolladores que tengan "JavaScript" como habilidad.
2. Extraer un array con TODOS los nombres de proyectos (sin duplicados).

Resultado esperado:

```
const javascriptDevelopers = [
  {
    id: 1,
    nombre: "Juan",
    habilidades: ["JavaScript", "HTML", "CSS"],
    proyectos: [
      { id: 1, nombre: "Proyecto 1" },
      { id: 2, nombre: "Proyecto 2" }
    ]
  },
  {
    id: 4,
    nombre: 'Miriam',
    habilidades: ['UX', 'Figma', 'HTML', 'JavaScript'],
    proyectos: [
      { id: 5, nombre: 'Proyecto 1' },
      { id: 6, nombre: 'Proyecto 4' }
    ]
  }
]

const projectsNames = ['Proyecto 1', 'Proyecto 2', 'Proyecto
3','Proyecto 4'];
```

EJERCICIO 4: Arreglar bug asíncrono

El cliente está intentando obtener datos de un usuario usando su ID (el ID 1 debería existir), pero **siempre obtiene `undefined`**. Nos ha pasado el código para que lo revisemos y arreglemos.

TAREAS:

1. Identifica el problema (¿por qué da undefined?)
2. Explica qué es la asincronía y por qué causa este problema
3. Arréglalo usando Promises
4. Arréglalo también usando async/await

Código a arreglar, simula una llamada a un API que trae usuarios de una base de datos:

```
function getUser(id) {
  let user;
  setTimeout(() => {
    if (id === 1) {
      user = { id: 1, name: "John Doe" };
    }
  }, 2000);
  return user;
}

const user = getUser(1);
console.log(user);
```

Resultado esperado:

```
Usuario: { id: 1, nombre: 'John Doe' }
```

EJERCICIOS A ELEGIR (puedes hacer el 5 o el 6):

EJERCICIO 5: Gestión de Taller

Retomamos el ejercicio del **parque de coches** que vimos en clase. De tal forma que vamos a ampliar las funciones para gestionar el inventario del taller.

Datos base: Usa el modelo de datos que creamos en clase (puedes simplificarlo si lo necesitas) y añade 5 coches más, con diferentes datos.

Funciones a implementar:

1. **`getCarsWithBeacon(cars):`**

   Retorna array con los coches que TIENEN baliza V16.

2. **`getCarsWithApprovedBeacon(cars):`**

   Retorna array con coches que tienen baliza V16 Y está homologada, útil para saber qué coches cumplen la normativa.

3. **`getCarsWithLowPressure(cars, minPressure):`**

   Recibe un mínimo de presión (ej: 2.0 bar). Retorna coches con AL MENOS UNA rueda por debajo del mínimo. Incluye información de qué ruedas están bajas

4. **`getCarReport(cars, licensePlate):`**

   Busca un coche por matrícula. Genera un informe legible en texto. Incluye: datos básicos (matrícula, marca, Kms, propietario…), estado de seguridad, ruedas, historial de visitas al taller.

Requisitos técnicos:

- Usa métodos de arrays (`.filter()`, `.map()`, `.find()`),
- NO mutes los datos originales
- Maneja casos donde no se encuentre el coche

TAREAS:
1. Array de datos con al menos 5 coches variados
2. Las 4 funciones implementadas
3. Ejemplos de uso de cada función
4. Comentarios explicando la lógica (si fuera necesario)

EJERCICIO 6: Validaciones con Regex

Crear un sistema completo de validación de formularios usando expresiones regulares (regex).

Función principal

Debes crear una función **`validateForm(data)`** que valide todos los campos de un formulario.

Campos a validar:
- **Email**: Formato válido, debe contener @ y terminar en *.loquesea*
- **Password**: Mínimo 8 caracteres, al menos una mayúscula, al menos una minúscula, al menos un número, al menos un caracter especial de estos: !@#$%^&*
- **Phone**: Asegurar que contiene +34. Debe empezar por 6 o 7. El número de teléfono debe tener 9 dígitos. Puede tener espacios, guiones o estar junto.
- **DNI**: Formato español: 12345678A, 8 dígitos seguidos de 1 letra mayúscula
- **Birthdate**: Formato DD/MM/YYYY, Debe ser una fecha válida. La persona debe ser mayor de 18 años.

Ejemplo de formato de entrada:

```
const formData = {
  email: "ana@example.com",
  password: "Pass123!",
  phone: "+34 612 345 678",
  dni: "12345678A",
```

```
  birthdate: "15/03/1990"
};
```

Salida esperada:

```
{
  valid: true, // o false si hay errores
    errors: [
      // Array vacío si todo OK, o lista de errores:
      { field: "email", message: "Formato de email inválido" },
      { field: "password", message: "La contraseña debe tener al menos
8 caracteres" },
      // ... más errores
    ],
    data: {
    email: "ana@example.com",
    emailNormalized: "ana@example.com", // lowercase, trimmed
    phone: "+34 612345678", // normalizado
    age: 35, // calculado desde birthdate
    // ... más datos procesados
  }
}
```

TAREAS:

1. Función principal **`validateForm()`**
2. Funciones auxiliares para cada campo
3. Casos de prueba
4. Comentarios en las funciones si procede

Ejercicio MINI-HOTEL (Opcional)

NOTA: Este ejercicio es completamente **OPCIONAL**. Hazlo si has completado los anteriores y quieres un desafío extra.

Contexto:

Un hotel nos contrata para crear un sistema completo de gestión de reservas. Debe manejar búsqueda de disponibilidad, creación de reservas, check-in, check-out, facturación y reportes.

PARTE 1: ESTRUCTURA DE DATOS PROPUESTA

Hotel:

```
{
  name: "Hotel Keepcoding",
  rooms: [...], // Array de habitaciones
  reservations: [...], // Reservas activas
  history: [...] // Reservas completadas
}
```

Habitación:

```
{
  number: 101,
  type: "single", // "single", "double", "suite"
  pricePerNight: 50,
  status: "available", // "available", "occupied", "maintenance"
  features: ["wifi", "tv", "minibar"]
}
```

Reserva:

```
{
  id: "RES-001",
  roomNumber: 101,
  guest: {
    name: "Ana García",
    email: "ana@example.com",
    phone: "+34 612345678",
    dni: "12345678A"
  },
  checkIn: "2026-01-20", // YYYY-MM-DD
  checkOut: "2026-01-25",
  nights: 5, // Calculado automáticamente
  totalPrice: 250, // Precio total
  status: "confirmed", // "confirmed", "checked-in", "checked-out",
"cancelled"
  extras: [
    { name: "Breakfast", price: 10, quantity: 5 },
    { name: "Parking", price: 15, quantity: 5 }
  ]
}
```

PARTE 2: FUNCIONES REQUERIDAS

Función 1: `createHotel(name, rooms);`
Crea el objeto hotel inicial.

Ejemplo de entrada:

```
const rooms = [
  { number: 101, type: "single", pricePerNight: 50, status:
"available", features: ["wifi", "tv"] },
  { number: 102, type: "single", pricePerNight: 50, status:
"available", features: ["wifi", "tv"] },
  { number: 201, type: "double", pricePerNight: 80, status:
"available", features: ["wifi", "tv", "minibar"] }
  // Añade más habitaciones
];

const hotel = createHotel("Hotel Keepcoding", rooms);
```

Salida:

```
{
  name: "Hotel Keepcoding",
  rooms: [...],
  reservations: [],
  history: []
}
```

Función 2: `searchAvailableRooms(hotel, checkIn, checkOut, roomType);`
Busca y devuelve habitaciones disponibles para unas fechas específicas. Hay que tener en cuenta que una habitación NO está disponible si existe una reserva en curso (ya sea porque la reserva esté activa, o las fechas se solapan).

Parámetros:

- hotel: objeto hotel
- checkIn: fecha de entrada (string "YYYY-MM-DD") prevista
- checkOut: fecha de salida (string "YYYY-MM-DD") prevista
- roomType: tipo de habitación (opcional, puede ser null)

Retorna: Array de habitaciones disponibles

Función 3: `createReservation(hotel, roomNumber, guestData, checkIn, checkOut);`
Crea una nueva reserva.

Validaciones:

- La habitación debe existir y estar disponible para esas fechas
- checkIn no puede ser en el pasado
- checkOut debe ser posterior a checkIn
- Mínimo 1 noche, máximo 14 noches
- Validar email, DNI y teléfono con regex

Cálculos automáticos:

- Número de noches: diferencia de fechas
- Precio total: noches * precioPorNoche
- ID único de reserva: RES-${timestamp}-${random}

Retorna: Objeto de reserva creado o error

Función 4: `addExtras(hotel, reservationId, extras);`
Añade servicios extras a una reserva existente

Entrada:

```
addExtras(hotel, "RES-XXXX", [
  { name: "Breakfast", price: 10, quantity: 5 },
  { name: "Parking", price: 15, quantity: 5 }
]);
```

Efecto:

- Añade los servicios al array de extras de la reserva
- Recalcula el precio total

Retorna: la reserva actualizada

Función 5: `checkIn(hotel, reservationId);`
Realiza el check-in de una reserva.

Efecto:

- Cambia estado de reserva a "checked-in"
- Cambia estado de habitación a "occupied"
- Registra fecha de check-in real

Validaciones:

- La reserva debe existir
- El estado debe ser “confirmed”
- La fecha debe ser igual o posterior al checkIn previsto

Retorna: mensaje de confirmación

Función 6: `checkOut(hotel, reservationId);`
Realiza el check-out y genera factura.

Efecto:
- Cambia estado de reserva a “checked-out”
- Cambia estado de habitación a "available"
- Mueve reserva de **`reservations`** a **`history`**
- Genera y retorna factura detallada

Factura debe incluir:

- ID de reserva
- Datos del huésped
- Habitación y tipo
- Fechas de estancia
- Desglose: habitación, extras, total
- Fecha de emisión

Función 7: `cancelReservation(hotel, reservationId);`
Cancela una reserva, debes tener en cuenta la fecha actual en la que se hace la petición de cancelación para cumplir la siguiente política:

Política de cancelación:
- Más de 7 días antes del check-in: sin cargo
- Entre 3-7 días: 50% del total
- Menos de 3 días: 100% del total

Efecto:
- Cambia estado de reserva a “cancelled”
- Libera la habitación
- Calcula y retorna penalización
- Mueve la reserva al historial

Retorna: un mensaje con la confirmación de cancelación y el importe de penalización

Función 8: `getOccupancyReport(hotel, date);`
Genera un reporte de ocupación para una fecha (pasada, presente o futura). Debes contar el total de habitaciones, verificar cuáles están ocupadas y calcula el porcentaje de ocupación.

PISTA: Al generar el informe desde el objeto *reservations*, puedes ver la ocupación de cualquier día. Una habitación está ocupada en una fecha si existe una reserva donde:

```
// La habitación está ocupada si: checkIn <= date < checkOut
checkIn <= fecha && fecha < checkOut
```

Retorna:

```
{
  date: "2026-01-20",
  total: 10,
  occupied: 6,
  available: 4,
  percentage: 60,
  byType: {
    single: { total: 4, occupied: 2 },
    double: { total: 4, occupied: 3 },
    suite: { total: 2, occupied: 1 }
  }
}
```

Función 9: `getRevenueReport(hotel, startDate, endDate);`
Calcula ingresos entre fechas

Retorna:

```
{
  period: "2026-01-01 to 2026-01-31",
  rooms: 5000,
  extras: 1200,
  total: 6200,
  reservations: 25
}
```

Función 10: `getGuestHistory(hotel, guestEmail);`
Busca todas las reservas de un cliente por email

Retorna: Información del Array de reservas (activas + historial) ordenadas por fechas

```
{
  guestEmail: "ana@example.com",
  guestName: "Ana García",
  totalReservations: 3,
  totalSpent: 1200,
  reservations: [
   {
     id: "RES-001",
     status: "checked-out",
     checkIn: "2025-08-15",
     checkOut: "2025-08-20",
     totalPrice: 525
```

```
    },
    {
      id: "RES-028",
      status: "checked-out",
      checkIn: "2025-12-20",
      checkOut: "2025-12-25",
      totalPrice: 850
     },
    //...
  ]
}
```

PARTE 3: REGLAS DE NEGOCIO

Precios por tipo:

- Single: €50/noche
- Double: €80/noche
- Suite: €150/noche

Servicios extra disponibles:

- Desayuno: €10/día
- Parking: €15/día
- Late check-out: €30 (cargo único)
- Early check-in: €20 (cargo único)

EJEMPLO DE USO

```
// Crear hotel
const hotel = createHotel("Hotel Keepcoding", [
  { number: 101, type: "single", pricePerNight: 50, status:
"available", features: ["wifi", "tv"] },
  { number: 201, type: "double", pricePerNight: 80, status:
"available", features: ["wifi", "tv", "minibar"] },
  { number: 301, type: "suite", pricePerNight: 150, status:
"available", features: ["wifi", "tv", "minibar", "jacuzzi"] }
]);

// Buscar disponibilidad
const available = searchAvailableRooms(hotel, "2026-01-20",
"2026-01-25", "double");
```

```
console.log(`${available.length} habitaciones disponibles`);

// Crear reserva
const reservation = createReservation(
  hotel,
  201,
  {
    name: "Ana García",
    email: "ana@example.com",
    phone: "+34 612345678",
    dni: "12345678A"
  },
  "2026-01-20",
  "2026-01-25"
);

console.log(`Reserva creada: ${reservation.id}`);
console.log(`Total: €${reservation.totalPrice}`);

// Añadir extras
addExtras(hotel, "RES-XXXX", [
  { name: "Breakfast", price: 10, quantity: 5 },
  { name: "Parking", price: 15, quantity: 5 }
]);

console.log(`Nuevo total: €${reservation.totalPrice}`);

// Check-in
checkIn(hotel, "RES-XXXX");
console.log("Check-in realizado");

// Check-out
const invoice = checkOut(hotel, "RES-XXXX");
console.log("Factura:", invoice);

// Cancelación de reserva
const cancellation = cancelReservation(hotel, "RES-XXXX");
console.log("Cancelación confirmada:", cancellation);

// Reporte de ocupación
const report = getOccupancyReport(hotel, "2026-01-20");
console.log("Ocupación:", report);
```

```
// Reporte de ingresos
const revenue = getRevenueReport(hotel, "2026-01-01", "2026-01-31");
console.log("Ingresos:", revenue);

// Historial de cliente
const guestHistory = getGuestHistory(hotel, "ana@example.com");
console.log("Historial de cliente:", guestHistory);
```

Si llegado hasta aquí has, mi más sincera enhorabuena te doy…